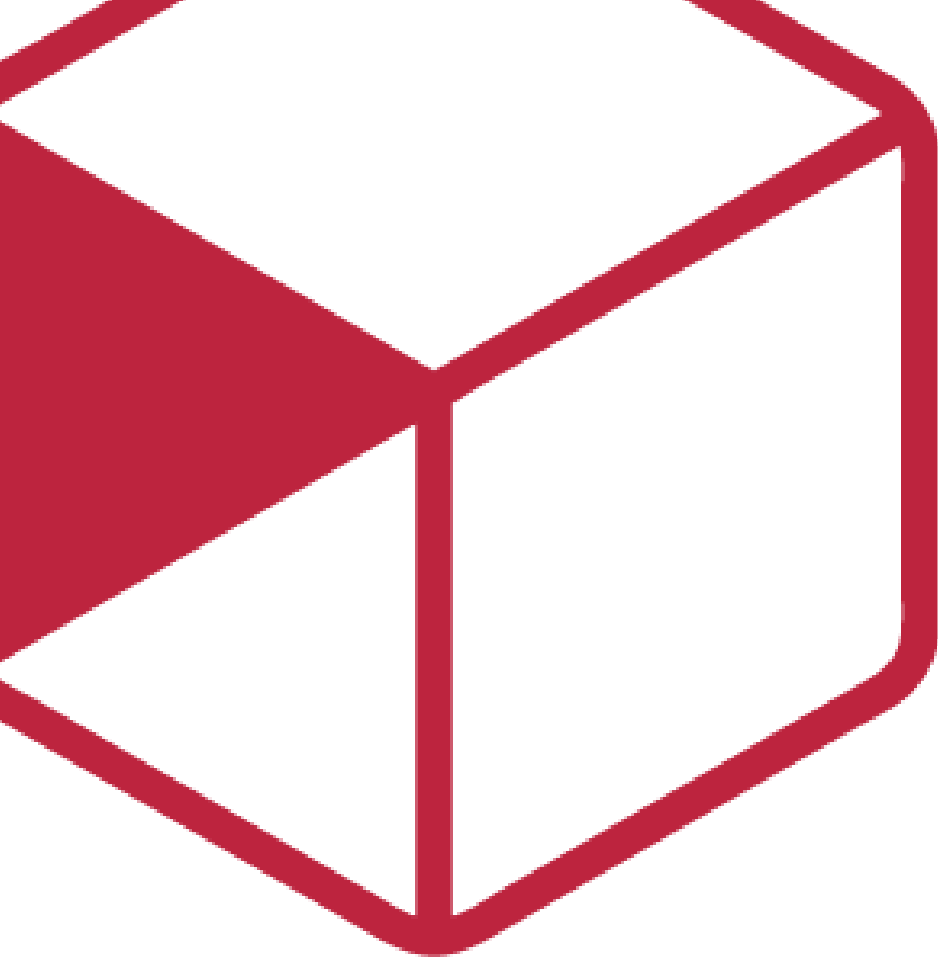




Diseño de la solución

Requisito R16A-RE-FU-003.

FORMATO	Arquitectura
PROYECTO	R16 - Adquisiciones
REFERENCIA	AUI- FOR-01
VERSIÓN	1.3
FECHA	30/06/2026
AUTOR	Isai Amaury Garcia Flores
REVISOR	Valdemar Farina Sanchez

Contenido

Importante	3
Introducción	4
1. Propósito del documento	4
2. Alcance	4
Específicamente incluye:	4
No se consideran:	5
**En caso de aplicar colocar lista de consideraciones que no aplicaran	5
Visión general del diseño	6
1. Objetivo técnico	6
2. Componentes involucrados	6
Diseño funcional detallado	7
1. Carga inicial y filtrado del historial	7
2. Criterios de aceptación del requisito	8
3. Reglas técnicas aplicadas	9
Diseño de componentes	10
1. Diagramas	10
Impacto Técnico	11
1. Impacto en código existente	11
2. Impacto en modelos por actualización de modelos	11
Manejo de Errores y Excepciones	12
Estrategia de Pruebas (Diseño de las pruebas)	12
1. Pruebas funcionales (Criterios de Aceptación en DEV)	13
2. Pruebas técnicas(unitarias e integración)	13
a. Unitarias	13
b. Pruebas de integración	13
3. Casos críticos	13
Control de versiones	14



Importante

Posterior a este diseño, ¿cómo saber si el diseño de la solución al requisito está completo para que el programador inicie con el desarrollo?

Hazte estas preguntas rápidas:

- ¿El programador sabe qué flujo implementar?
- ¿Sabe qué pasa si algo falla?
- ¿Sabe qué reglas no puede romper?
- ¿Sabe qué pruebas debe pasar?
- ¿Sabe dónde impacta?

Nota: Este documento es una propuesta basada en el estándar IEEE 1016 “Software Design Description” que va permitir abarcar los puntos más importantes para el diseño del requisito que se está trabajando. Se debe administrar el tiempo que se tiene de diseño para que se complete de la mejor manera considerando todos los detalles técnicos.



Introducción

1. Propósito del documento

Definir el diseño técnico backend para la sección Documentos Regulatorios del Catálogo de Clientes, sobre el modelo genérico de archivos por dominio (ArchivoDominio / catDominioEntidad / CatalogoTipoDocumento), **encapsulado detrás de una capa de adaptación específica de Cliente** que mantiene estable el contrato consumido por Frontend.

2. Alcance

Específicamente incluye:

- Adopción del modelo genérico ArchivoDominio + catDominioEntidad + CatalogoTipoDocumento como capa de persistencia.
- Capa de adaptación específica de Cliente (DocumentosRegulatoriosClienteController / DocumentosRegulatoriosClienteBO) que conserva ruta y forma de payload de los 4 endpoints ya consumidos por Front (consulta, carga, eliminación, descarga), delegando internamente en los BOs genéricos.
- Endpoint DELETE explícito de documento regulatorio.
- Enforcement del rol Regulatorios (Usuario.GestorDeAsuntosRegulatorios) en backend, dentro de la capa de adaptación de Cliente.
- Validación de formato PDF en backend antes de cualquier escritura en BD o MinIO, ahora data-driven vía CatalogoTipoDocumento.FormatoAceptado en lugar de hardcodeada
- Endpoints genéricos de infraestructura (POST /Archivo/Subir, PUT /ArchivoDominio, POST /ArchivoDominio/QueryResult, GET /ArchivoDominio/{id}/Pdf) — quedan disponibles como capacidad de plataforma para futuros dominios (Proveedor, Producto); la pantalla de Documentos Regulatorios de Cliente no los consume directo, consume la capa de adaptación
- Endpoint nuevo GET /Validar/Cliente/{idCliente}/DocumentosRegulatorios (patrón Chain of Validators) para que Pretramitar Pedido deje de leer la tabla directo
- Refuerzo a nivel BD de la Regla R3 mediante índice único filtrado, ahora sobre ArchivoDominio

No se consideran:

- Consumo directo de los endpoints genéricos (/Archivo/Subir, /ArchivoDominio/*) desde el Frontend del Catálogo de Clientes — la pantalla sigue hablando con la capa de adaptación de Cliente



- Adopción del modelo genérico para Proveedor o Producto — la propuesta lo habilita estructuralmente, pero no se implementan esos dominios en este requisito
- Flujo de upload por URL pre-firmada / UriSubirArchivo + ArchivoSubirCrudoBO (el upload sigue siendo server-side, ahora en dos pasos internos: Subir genérico + Vincular a dominio, orquestados por la capa de adaptación en una sola llamada para Front).
- Soporte de formatos distintos a PDF, en backend o frontend
- Implementación de validadores adicionales en el endpoint Validar más allá de ValidadorDocumentosRegulatorios
- Purga de archivos físicos en MinIO (política de limpieza fuera de scope)
- Gestión del rol Regulatorios (se asigna a nivel BD, vía Usuario.GestorDeAsuntosRegulatorios)
- Clientes de Región Perú (no implementado en esta release)

Visión general del diseño

1. Objetivo técnico

Exponer los endpoints necesarios para que el frontend pueda gestionar los documentos regulatorios de un cliente (Licencia Sanitaria y Aviso de Responsable Sanitario), sobre el modelo genérico ArchivoDominio (vínculo entidad-de-dominio↔archivo↔tipo-de-documento), CatalogoTipoDocumento (tipo de documento, scopeado por dominio, con metadata de UI) y catDominioEntidad (catálogo de dominios: Cliente / Proveedor / Producto).

2. Componentes involucrados

Identificar **componentes reales** del sistema que participan e indicar la responsabilidad de cada uno.

Aplicativo	Componente	Responsabilidad	Ubicación
Backend	DocumentosRegulatoriosClienteController (adaptación de ArchivoClienteController)	4 endpoints REST estables para FE (consulta, carga, eliminar, Pdf)	WebApi.Catalogos\Controllers\Configuracion\Clientes\Relaciones\ (existente, refactor interno)
Backend	DocumentosRegulatoriosClienteBO (adaptación de ArchivoClienteBO.Do)	Resuelve claves de tipo de documento del dominio Cliente, valida rol	Logic.Pqf.Catalogos\Clientes\Relaciones\ (existente, refactor interno)



	cumentosRegulatorio s.cs)	Regulatorios y PDF, orquestra ArchivoBO.Subir + ArchivoDominioBO	
Backend	ArchivoDominioBO	CRUD genérico + lógica de reemplazo atómico + validación de dominio polimorfo	Logic.Pqf.Catalogos\ Dominio\ (nuevo)
Backend	CatalogoTipoDocume ntoBO	CRUD genérico del catálogo de tipos de documento	Logic.Pqf.Catalogos\ Dominio\ (nuevo)
Backend	catDominioEntidadB O	CRUD genérico del catálogo de dominios	Logic.Pqf.Catalogos\ Dominio\ (nuevo)
Backend	ArchivoBO.Subir	Sube archivo a MinIO y registra Archivo, genérico (independiente de dominio)	Logic.Pqf.Catalogos\ Archivos\ArchivoBO.c s (nuevo método)
Backend	ArchivoBO.Descargar ArregloBytes	Descarga bytes del PDF desde MinIO para entrega inline	Logic.Pqf.Catalogos\ Archivos\ArchivoBO. DownloadExtensions. cs (existente, sin cambios)
Backend	ArchivoDominioContr oller	Endpoints genéricos de infraestructura (Subir/Vincular/Query /Pdf) — capacidad de plataforma, no consumida por esta pantalla	WebApi.Catalogos\ ontrollers\Dominio\ (nuevo)
Backend	IValidadorCliente / ValidadorDocumento sRegulatorios	Patrón Chain of Validators para el endpoint Validar	Logic.Pqf.Catalogos\ Validacion\ (nuevo)
Backend	ValidacionClienteCon troller	GET /Validar/Cliente/{idCli ente}/DocumentosRe gulatorios	WebApi.Catalogos\ ontrollers\Validacion\ (nuevo)



BD	ArchivoDominio	Vincula entidad de dominio (Cliente/Proveedor/Producto) + Archivo + CatalogoTipoDocumento; Activo=1 = vigente	ProquifaDotNetEntities (nuevo)
BD	CatalogoTipoDocumento	Catálogo de tipos de documento, scopeado por dominio, metadata UI	ProquifaDotNetEntities (nuevo)
BD	catDominioEntidad	Catálogo de tipos de entidad de dominio	ProquifaDotNetEntities (nuevo)
BD	ArchivoCliente / catUsoArchivoSistema	Modelo previo — deprecado, solo lectura durante la transición	ProquifaDotNetEntities (existente)
BD	Usuario.GestorDeAsuntosRegulatorios	Campo bit que habilita el rol Regulatorios	ProquifaDotNetEntities (existente, sin cambios)

Diseño funcional detallado

1. Modelo de datos

Entidades reutilizadas (sin tabla nueva)

```

catDominioEntidad (Cliente / Proveedor / Producto / ...)
  └─ 1:N
CatalogoTipoDocumento (LicenciaSanitaria, AvisoResponsableSanitario, ... -- scopeado por dominio)
  └─ 1:N
ArchivoDominio (vínculo archivo ↔ entidad de dominio; Activo=1 = vigente)

```



```

└─ FK IdArchivo
Archivo (FileKey, FileBucket → MinIO --
existente, sin cambios)

```

```

-- 1) Seed del catálogo (bloqueante para todo lo demás -- GAP-05)
INSERT INTO dbo.catUsoArchivoSistema (IdCatUsoArchivoSistema, Clave,
Descripcion, Activo)
VALUES
    (NEWID(), 'LicenciaSanitaria', 'Licencia Sanitaria',
1),
    (NEWID(), 'AvisoResponsableSanitario', 'Aviso de Responsable
Sanitario', 1);

-- 2) Refuerzo a nivel BD de Regla R3 (una versión vigente por tipo por
cliente)
-- ArchivoCliente actualmente NO tiene este constraint; depende solo de
la
-- lógica de aplicación en ArchivoClienteB0. Se agrega como defensa
adicional.
CREATE UNIQUE INDEX UX_ArchivoCliente_ClienteUso
ON ArchivoCliente (IdCliente, IdCatUsoArchivoSistema)
WHERE Activo = 1;

```

Decisiones de diseño

Decisión	Justificación
Adoptar el modelo genérico ArchivoDominio/catDominioEntidad/CatalogoTipoDocumento del líder backend	Evita duplicar 3 tablas + 4 endpoints cuando Proveedor/Producto necesiten documentos por dominio; metadata de UI (formato, orden, tamaño) queda en BD en vez de hardcodeada



Capa de adaptación específica de Cliente, en vez de exponer el modelo genérico directo a Front	El Front del Catálogo de Clientes ya está construido contra el contrato de la versión actual en sistema; cambiar el modelo de persistencia no debe forzar un segundo rework de FE. La capa de adaptación absorbe el cambio
Rol GestorDeAsuntosRegulatorios y DELETE explícito viven en la capa de adaptación de Cliente, no en ArchivoDominioBO	Son reglas de negocio específicas de "Documentos Regulatorios de Cliente". Acoplarlas al BO genérico contradice el objetivo de reutilización entre dominios (Proveedor/Producto pueden tener reglas de autorización distintas)

2. Diseño de business objects

Estructura de archivos

```

Logic.Pqf.Catalogos\Clientes\Relaciones\
├── DocumentosRegulatoriosClienteBO.cs           (nuevo -- reemplaza
ArchivoClienteBO.DocumentosRegulatorios.cs)
├── Models\
│   ├── DocumentoRegulatorioDetalle.cs
│   └── DocumentosRegulatoriosCliente.cs

Logic.Pqf.Catalogos\Dominio\
├── ArchivoDominioBO.cs                         (nuevo -- genérico)
├── CatalogoTipoDocumentoBO.cs                 (nuevo -- genérico)
└── catDominioEntidadBO.cs                     (nuevo -- genérico)

Logic.Pqf.Catalogos\Validacion\
├── IValidadorCliente.cs                       (nuevo)
├── ResultadoRegla.cs                          (nuevo)
└── ValidadorDocumentosRegulatorios.cs         (nuevo)

```

ArchivoClienteBO.cs y ArchivoClienteController.cs (las clases base, no la partial de Documentos Regulatorios) quedan deprecadas, vivas solo para otros usos no migrados de ArchivoCliente durante la transición.



Clase: DocumentoRegulatorioDetalle

```
public sealed class DocumentoRegulatorioDetalle
{
    public Guid IdArchivoCliente { get; set; } // mapea a
    ArchivoDominio.IdArchivoDominio -- nombre conservado por estabilidad de
    contrato (ver Decisiones de diseño)
    public Guid IdArchivo { get; set; }
    public DateTime FechaRegistro { get; set; }
}
```

Clase: DocumentosRegulatoriosCliente (respuesta del endpoint de consulta)

```
public sealed class DocumentosRegulatoriosCliente
{
    public DocumentoRegulatorioDetalle LicenciaSanitaria { get;
    set; } // null si no cargado
    public DocumentoRegulatorioDetalle AvisoResponsableSanitario { get;
    set; } // null si no cargado
}
```

Lógica de archivoDominioBO

```
_GuardarOActualizar(idArchivo, idCatalogoTipoDocumento,
idDominioEntidadArchivo):
1. Validar que CatalogoTipoDocumento.IdcatDominioEntidad corresponda al
dominio del idDominioEntidadArchivo
   (ej. si el tipo aplica a Cliente, el GUID debe existir en Cliente)
2. Validar que FormatoAceptado del CatalogoTipoDocumento incluya el
mimeType del Archivo referenciado
3. Validar TamanoMaximoKB contra el tamaño del archivo
4. Buscar ArchivoDominio activo con mismo (IdDominioEntidadArchivo,
IdCatalogoTipoDocumento)
5. Si existe → SET Activo = 0
6. INSERT nuevo ArchivoDominio con Activo = 1
```

Logica de DocumentosRegulatoriosClienteBO



Operacion: Obtener documentos regulatorios del cliente

Entrada: IdCliente (Guid)

Salida: DocumentosRegulatoriosCliente

Pasos:

1. Resolver IdCatalogoTipoDocumento de 'LicenciaSanitaria' y 'AvisoResponsableSanitario' para el dominio Cliente (cache en memoria, no por request)
2. Consultar ArchivoDominio WHERE IdDominioEntidadArchivo = @idCliente AND Activo = 1
AND IdCatalogoTipoDocumento IN (...)
3. INNER JOIN Archivo para FechaRegistro
4. Mapear por tipo; si no existe uno, esa clave queda en null (nunca se omite)

Operacion: VArgar documento regulatorio (cargar o reemplazar)

Entrada: IdCliente (Guid), Clave tipo documento ("LicenciaSanitaria"|"AvisoResponsableSanitario"),
archivo binario (multipart), IdUsuarioAutenticado
Salida: DocumentoRegulatorioDetalle

Precondiciones (validar ANTES de tocar BD o MinIO):

1. IdUsuarioAutenticado.GestorDeAsuntosRegulatorios = 1 → si no, 403
2. Content-Type/extensión del archivo = PDF → si no, 400 (sin escritura)

Pasos (transacción):

1. Resolver IdCatalogoTipoDocumento a partir de la Clave recibida (dominio Cliente)
2. ArchivoBO.Subir(file, bucket, region) → sube a MinIO, INSERT en Archivo, obtiene IdArchivo (genérico, Tarea 2 de la propuesta)
3. ArchivoDominioBO._GuardarOActualizar(idArchivo, idCatalogoTipoDocumento, idCliente) → aplica reemplazo atómico (genérico, Tarea 1)
4. Retornar DocumentoRegulatorioDetalle del nuevo registro (mapeado, ver Decisiones de diseño sobre naming del campo)

Nota: el Front sigue enviando un único multipart en un solo request; los pasos 2-3 son internos del backend (no se exponen como 2 llamadas HTTP para esta



```
pantalla).
```

Operacion: Obtener PDF para visualización

```
Entrada: IdArchivoCliente (Guid → IdArchivoDominio internamente)
```

```
Salida: byte[] + Content-Type + Content-Disposition
```

Pasos:

1. Obtener ArchivoDominio WHERE IdArchivoDominio = @id AND Activo = 1 → 404 si no existe/inactivo
2. Obtener IdArchivo asociado
3. ArchivoBO.DescargarArregloBytes(idArchivo) → bytes desde MinIO
4. Retornar HttpResponseMessage: Content-Type: application/pdf, Content-Disposition: inline; filename="documento.pdf"

Diagramas

Diagrama de Tablas Asociadas

Modelo de datos vigente (catDominioEntidad / CatalogoTipoDocumento / ArchivoDominio / Archivo) junto con el modelo previo, deprecado pero aún vivo en modo solo lectura durante la transición (catUsoArchivoSistema / ArchivoCliente). Cliente se incluye solo como referencia del vínculo polimorfo (IdDominioEntidadArchivo, sin FK física).

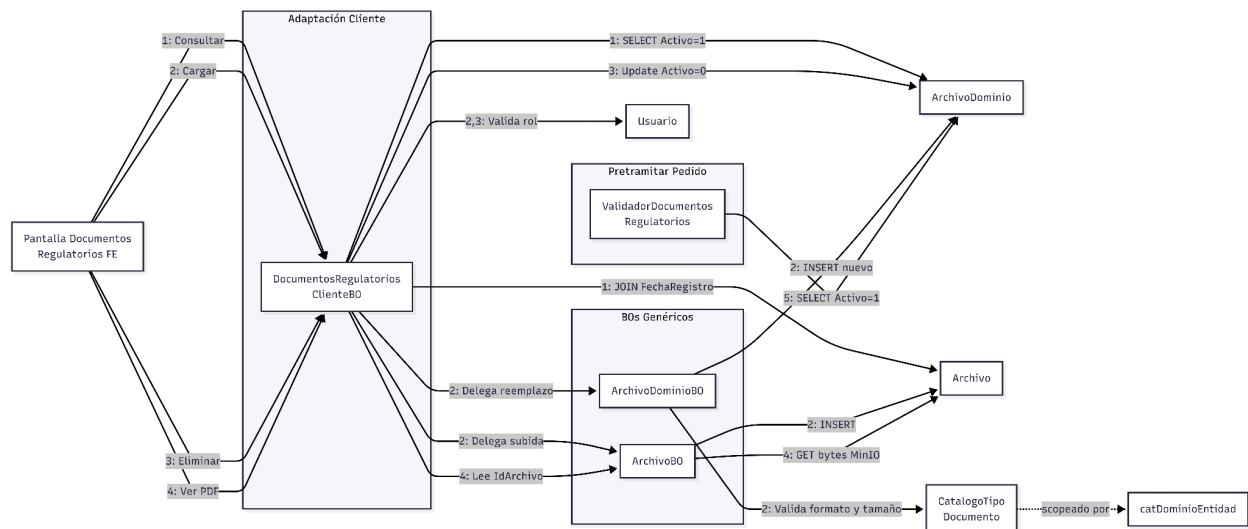




Diagrama de funcionalidad

Qué hace cada capa de Business Object con cada tabla, por operación (numeración alineada a los Endpoints 1-5 de Contratos de API).





Contratos de API

Controller: ArchivoClienteController (existente, extendido)

Prefijo de ruta: /ArchivoCliente

Endpoint 1 — Obtener documentos regulatorios del cliente

```
GET /ArchivoCliente/DocumentosRegulatorios/{idCliente}
```

Response 200:

```
{
  "licenciaSanitaria": {
    "idArchivoCliente": "3fa85f64-...",
    "idArchivo": "7c9e6679-...",
    "fechaRegistro": "2026-06-16T10:00:00"
  },
  "avisoResponsableSanitario": null
}
```

Endpoint 2 — Cargar documento regulatorio (carga o reemplazo)

```
POST /ArchivoCliente/DocumentoRegulatorio/{idCliente}
Content-Type: multipart/form-data
```



tipoDocumentoRegulatorio $\in$ { "LicenciaSanitaria", "AvisoResponsableSanitario" }. El backend resuelve internamente la clave al IdCatalogoTipoDocumento del dominio Cliente.

Response 200:

```
{
  "idArchivoCliente": "3fa85f64-...",
  "idArchivo": "7c9e6679-...",
  "fechaRegistro": "2026-06-16T10:05:00"
}
```

Response 400 — archivo no es PDF, o tipoDocumentoRegulatorio no es una clave válida:

```
{ "mensaje": "Solo se aceptan documentos en formato PDF." }
```

Response 403 — usuario sin GestorDeAsuntosRegulatorios = 1:

```
{ "mensaje": "No tiene permisos para cargar documentos regulatorios." }
```

Endpoint 3 — Eliminar documento regulatorio

```
DELETE /ArchivoCliente/DocumentoRegulatorio/{idCliente}/{idArchivoCliente}
```

Response 200:

```
{ "eliminado": true }
```

Response 404 — registro no existe o ya está inactivo:

```
{ "mensaje": "Documento regulatorio no encontrado." }
```

Response 403 — usuario sin rol Regulatorios, o registro pertenece a otro cliente:

```
{ "mensaje": "No tiene permisos para eliminar este documento." }
```

Endpoint 4 — Obtener PDF para visualización

```
GET /ArchivoCliente/DocumentoRegulatorio/{idArchivoCliente}
```

Response 200: binario, Content-Type: application/pdf, Content-Disposition: inline; filename="documento.pdf"



Response 404 — registro no existe o está inactivo:

```
{ "mensaje": "Documento regulatorio no encontrado." }
```

Endpoint 5 — Validar cumplimiento de documentos regulatorios

```
GET /Validar/Cliente/{idCliente}/DocumentosRegulatorios
```

Response 200:

```
{
  "esValido": true,
  "reglas": [
    {
      "clave": "DOCUMENTOS_REGULATORIOS",
      "cumple": true,
      "detalle": [
        { "clave": "LicenciaSanitaria", "cumple": true,
      "idArchivoDominio": "guid" },
        { "clave": "AvisoResponsableSanitario", "cumple": false,
      "idArchivoDominio": null }
      ],
      "mensaje": "Falta cargar Aviso de Responsable Sanitario"
    }
  ]
}
```

Solucion / Proyecto	EndPoint	Parametros	Salida
Consultar Documentos Regulatorios	GET /ArchivoCliente/DocumentosRegulatorios/{idCliente}	idCliente (path)	JSON con licenciaSanitaria + avisoResponsableSanitario (ambas claves siempre presentes, null si no cargadas)
Cargar / Reemplazar Documento	POST /ArchivoCliente/Docu	idCliente, tipoDocumentoRegulatori	JSON DocumentoRegul



	mentoRegulatorio/{id Cliente}/{tipoDocumentoRegulatorio}	o (path); archivo binario (multipart/form-data)	atorioDetalle (idArchivoCliente, idArchivo, fechaRegistro); 400 si no-PDF; 403 si sin rol Regulatorios
Eliminar Documento	DELETE /ArchivoCliente/DocumentoRegulatorio/{id Cliente}/{idArchivoCliente}	idCliente, idArchivoCliente (path)	JSON { "eliminado": true }; 404 si no existe; 403 si sin rol o pertenencia distinta
Obtener PDF Visualización	GET /ArchivoCliente/DocumentoRegulatorio/{id ArchivoCliente}/Pdf	idArchivoCliente (path)	Binario application/pdf, Content-Disposition: inline; 404 si no existe/inactivo
Validar Cumplimiento Regulatorio	GET /Validar/Cliente/{idCliente}/DocumentosRegulatorios	idCliente (path)	JSON { esValido, reglas[] } con detalle de cada tipo; consumido por Pretramitar Pedido (R16A-RE-FU-009), no por esta pantalla

3. Impacto en código existente

Modificaciones

Archivo / Componente	Cambio
ArchivoClienteBO.DocumentosRegulatorios.cs → DocumentosRegulatoriosClienteBO.cs	Deja de tocar ArchivoCliente directo; delega en ArchivoDominioBO / ArchivoBO.Subir. Conserva firma pública (misma entrada/salida) — el controller no cambia



ArchivoClienteController (Endpoints 1-4)	Sin cambio de ruta ni contrato externo; internamente invoca DocumentosRegulatoriosClienteBO
ArchivoBO.cs	Nuevo método Subir(file, bucket, region) genérico, generaliza SubirArchivoDesdePeticion
Script de BD	3 tablas nuevas (catDominioEntidad, CatalogoTipoDocumento, ArchivoDominio) + migración + índice único filtrado, ahora en ArchivoDominio (ya no en ArchivoCliente)

Componentes nuevos

```

Logic.Pqf.Catalogos\Clientes\Relaciones\
├── DocumentosRegulatoriosClienteBO.cs
│   └── Models\
│       ├── DocumentoRegulatorioDetalle.cs
│       └── DocumentosRegulatoriosCliente.cs
Logic.Pqf.Catalogos\Dominio\
├── ArchivoDominioBO.cs
├── CatalogoTipoDocumentoBO.cs
└── catDominioEntidadBO.cs
Logic.Pqf.Catalogos\Validacion\
├── IValidadorCliente.cs
├── ResultadoRegla.cs
└── ValidadorDocumentosRegulatorios.cs
WebApi.Catalogos\Controllers\Dominio\
├── ArchivoDominioController.cs
WebApi.Catalogos\Controllers\Validacion\
├── ValidacionClienteController.cs

```

4. Criterios de aceptación del requisito



Realizar un mapeo explícito de los criterios de aceptación (CA) que demuestre que TODOS están cubiertos.

Ejemplo:

CA	Descripción	Estado	Justificación
A1	Dado que un usuario abre el Catálogo de Clientes y consulta un cliente específico, Cuando se muestra la pantalla del cliente, Entonces deberá presentarse la sección "Documentos Regulatorios" con el listado de los dos documentos posibles (Licencia Sanitaria y Aviso de Responsable Sanitario), indicando para cada uno si tiene archivo cargado o si está vacío.	Cubierto ▾	Endpoint 1 retorna siempre las dos claves, null si no hay ArchivoDomini o activo; ver Contrato Endpoint 1, Respuesta 200
A2	Dado que un usuario con rol distinto de Regulatorios consulta la sección Documentos Regulatorios de un cliente, Cuando visualiza la pantalla, Entonces el sistema deberá presentar el listado de documentos cargados en modo bloqueado: el usuario puede ver qué documentos están registrados y consultar su contenido, pero no puede cargar, reemplazar ni eliminar.	Cubierto ▾	Enforced en backend Endpoints 2 y 3: validación GestorDeAsuntosRegulatorios = 1 antes de cualquier escritura, retorna 403; ver Lógica DocumentosRegulatoriosClienteBO - "Cargar" y "Eliminar"



B1	Dado que el usuario tiene rol Regulatorios y consulta la sección Documentos Regulatorios de un cliente, Cuando ejecuta la acción de cargar Licencia Sanitaria o Aviso de Responsable Sanitario, Entonces el sistema deberá permitir seleccionar un archivo PDF del equipo del usuario y, al confirmar, almacenarlo asociado al cliente y al tipo de documento correspondiente.	Cubierto ▾	<i>Endpoint 2, upload server-side en un solo paso para FE (internamente Subir + Vincular); ver Contrato Endpoint 2, Flujo de "Cargar documento regulatorio"</i>
B2	Dado que el rol Regulatorios intenta cargar un archivo en formato distinto a PDF, Cuando el sistema valida el archivo, Entonces deberá rechazar la operación, no almacenar el archivo y mostrar un mensaje claro indicando que solo se aceptan documentos en formato PDF.	Cubierto ▾	<i>Backend valida formato antes de cualquier escritura en Endpoint 2 (400), ahora vía CatalogoTipo Documento.Formatado; ver Lógica "Cargar documento regulatorio" Precondición 2</i>



C1	Dado que un usuario consulta un documento previamente cargado, Cuando ejecuta la acción de visualizar, Entonces el sistema deberá entregar el archivo PDF al navegador del usuario. El comportamiento posterior de apertura (pestaña nueva, pestaña actual o descarga directa) depende de la configuración del navegador, fuera del control del sistema.	Cubierto ▾	<i>Endpoint 4 entrega bytes inline desde ArchivoDominio/Archivo; ver Contrato Endpoint 4, Lógica "Obtener PDF para visualización"</i>
D1	Dado que el cliente ya tiene cargada una Licencia Sanitaria o un Aviso de Responsable Sanitario y el rol Regulatorios carga una nueva versión del mismo documento, Cuando el sistema procesa la operación, Entonces deberá registrar la nueva versión como vigente. En el listado de pantalla únicamente se muestra la versión vigente; la versión anterior deja de listarse.	Cubierto ▾	<i>ArchivoDominioBO._Guardar OActualizar + índice único filtrado en ArchivoDominio o garantizan una única versión activa por tipo</i>
D2	Dado que el usuario tiene rol Regulatorios y un documento está cargado para un cliente, Cuando ejecuta la acción de eliminar, Entonces el sistema deberá solicitar confirmación explícita al usuario y, al confirmar, retirar el documento del catálogo del cliente. Tras la eliminación, el documento aparece como no cargado en el listado de pantalla.	Cubierto ▾	<i>Endpoint 3 ejecuta soft-delete (Activo = 0) con validación de rol y pertenencia; confirmación explícita es responsabilidad de Frontend; ver Contrato Endpoint 3, Lógica "Eliminar documento"</i>



			regulatorio"
El	Dado que el rol Regulatorios cargó un documento PDF para un cliente, Cuando el sistema almacena el archivo, Entonces no deberá ejecutar ninguna validación de fecha de vigencia, contenido del documento ni autenticidad. El documento se considera vigente para fines del sistema por el solo hecho de estar cargado en el catálogo.	Cubierto ▾	<i>Ni DocumentosRegulatoriosClienteBO ni ArchivoDominioBO validan fecha/contenido; presencia de registro activo es única condición de Pretramitar Pedido</i>



Manejo de Errores y Excepciones

Qué errores se esperan y cómo debe responder el sistema. Por ejemplo:

Escenario	Comportamiento esperado
Usuario sin GestorDeAsuntosRegulatorios = 1 intenta cargar/reemplazar/eliminar	403, sin escritura
Archivo subido no es PDF	400, sin escritura en BD ni en MinIO
tipoDocumentoRegulatorio no es clave válida del catálogo	400 con mensaje descriptivo
CatalogoTipoDocumento sin los registros requeridos para el dominio Cliente (seed no aplicado)	500 — error de configuración; operación de carga/consulta no puede resolver el tipo
IdDominioEntidadArchivo no corresponde al dominio del CatalogoTipoDocumento (validación polimorfa de ArchivoDominioBO)	400/500 según capa — no debería ocurrir si la capa de adaptación de Cliente resuelve el tipo correctamente; defensivo
Registro no encontrado en DELETE o GET Pdf	404 con mensaje
DELETE de registro que pertenece a otro cliente	403 con mensaje
Fallo de BD en la transacción de carga (pasos 2-3 de "Cargar documento regulatorio")	Rollback → 500. El archivo ya subido a MinIO queda huérfano (riesgo documentado en el requisito)
Fallo de MinIO al descargar bytes para visualización	500 con mensaje genérico



Estrategia de Pruebas (Diseño de las pruebas)

- ☐ A1: GET DocumentosRegulatorios/{idCliente} de cliente sin documentos → ambas claves en null
- ☐ A1b: un solo tipo cargado → una clave con datos, la otra null
- ☐ A2: usuario sin GestorDeAsuntosRegulatorios intenta POST/DELETE → 403 (enforcement backend, en la capa de adaptación de Cliente)
- ☐ B1: POST DocumentoRegulatorio/{idCliente}/{tipo} con PDF válido → GET DocumentosRegulatorios refleja el nuevo documento
- ☐ B2: POST con archivo no-PDF → 400, sin escritura en BD ni MinIO
- ☐ D1: Registrar mismo tipo dos veces → solo un registro activo en ArchivoDominio; el anterior con Activo = 0
- ☐ D2: DELETE válido → registro inactivo → GET DocumentosRegulatorios devuelve null para ese tipo
- ☐ D2b: DELETE de registro ya inactivo → 404
- ☐ Pertenencia: DELETE con idCliente distinto al del registro → 403
- ☐ Visualización: GET .../Pdf → bytes con Content-Type: application/pdf y Content-Disposition: inline
- ☐ Validar (nuevo): GET /Validar/Cliente/{idCliente}/DocumentosRegulatorios → esValido=true con ambos documentos activos; false con detalle si falta alguno

Pruebas unitarias

- _GuardarOActualizar corregido: al insertar nuevo activo del mismo tipo, el anterior queda Activo = 0
- Cargar sin rol Regulatorios → excepción de autorización antes de tocar MinIO
- Cargar archivo no-PDF → excepción de validación antes de tocar MinIO
- Cargar en cliente sin documento previo → INSERT correcto, retorna detalle
- Eliminar con IdCliente incorrecto → excepción de autorización
- Obtener PDF visualización → llama ArchivoBO.DescargarArregloBytes con el IdArchivo correcto

Control de versiones

Versión	Fecha	Autor	Tipo de Cambio	Descripción del cambio	Aprobó
1.0	06/03/25	Roberto Baez ...	Creación ▾	Se crea	Rose Ríos ...



				plantilla genérica	
1.1	16/06/26	Alan Fernand...	Creación ▾	Se crea plantilla genérica para DIS	
1.2	19/06/26	Isai Amaury Gar...	Actualiz... ▾	Se documenta diseño de la solución	
1.3	30/06/26	Isai Amaury Gar...	Actualiz... ▾	Se actualiza documento tras ajustes en propuesta de diseño	

